

CineIQ

A Hybrid Explainable Movie Recommendation and Review
Analysis System

Kartik Khare

Department of Engineering Physics
Indian Institute of Technology Guwahati

May 18, 2026

Contents

1	The Motivation Behind CineIQ	2
2	Constructing the Dataset Universe	3
3	Building Semantic Intelligence	4
4	Collaborative Intelligence and Matrix Factorization	5
5	The Sentiment Problem	6
6	Adaptive Recommendation and Explainability	7
7	Deployment Engineering	8
8	Conclusion	9

Chapter 1

The Motivation Behind CineIQ

Recommendation systems today form the computational backbone of modern entertainment ecosystems. Platforms such as Netflix, Spotify and YouTube continuously attempt to answer a deceptively simple question:

“What would a user most likely enjoy next?”

Traditional systems generally rely either on semantic similarity between items or behavioral similarity between users. However, these approaches individually suffer from limitations. Pure collaborative systems struggle with cold-start problems, while pure content-based systems fail to capture evolving user preferences.

CineIQ was conceptualized as an attempt to combine:

- semantic understanding of cinema,
- collaborative behavioral intelligence,
- emotional interpretation,
- explainability,
- and adaptive recommendation logic

into a unified recommendation framework.

The project gradually evolved from a simple TF-IDF recommender into a mathematically grounded hybrid architecture integrating matrix factorization, contextual NLP and adaptive ensemble ranking.

Chapter 2

Constructing the Dataset Universe

The first challenge involved integrating multiple datasets with fundamentally different objectives.

The TMDB 5000 dataset primarily provided semantic information:

- movie overviews,
- genres,
- keywords,
- cast,
- crew,
- directors.

On the other hand, the MovieLens 20M dataset provided behavioral information through user ratings.

This introduced the first architectural problem:

TMDB and MovieLens use different identifier systems.

The bridge between both datasets was achieved using:

`link.csv`

which maps:

MovieLens movieId $\rightarrow$ TMDB tmdbId

This synchronization stage became essential because the final hybrid system required semantic and behavioral layers to operate on the same movie space.

Chapter 3

Building Semantic Intelligence

The semantic recommendation pipeline was built using TF-IDF vectorization.

For every movie, a textual “soup” was constructed:

$$\text{Soup} = \text{Genres} + \text{Keywords} + \text{Cast} + \text{Director} + \text{Overview}$$

The TF-IDF weighting scheme was defined as:

$$TFIDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

where:

$$TF(t, d) = \frac{f_{t,d}}{\sum_k f_{k,d}}$$

and

$$IDF(t, D) = \log \left(\frac{|D|}{1 + |\{d : t \in d\}|} \right)$$

Movie similarity was computed using cosine similarity:

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

Movies with smaller angular separation in vector space are considered semantically closer.

Chapter 4

Collaborative Intelligence and Matrix Factorization

The MovieLens dataset enabled the construction of a sparse user-movie interaction matrix:

$$R \in \mathbb{R}^{m \times n}$$

where:

- m represents users,
- n represents movies.

Initially, the project attempted to use the `scikit-surprise` library for collaborative filtering because of its integrated recommendation APIs and built-in SVD implementation.

However, the library produced compatibility issues with NumPy 2.x due to outdated compiled extensions.

Rather than downgrading the scientific stack, the architecture was redesigned around:

- `scipy.sparse`,
- `scipy.sparse.linalg.svds`.

The collaborative model therefore relied directly on Singular Value Decomposition:

$$R \approx U \Sigma V^T$$

where:

- U stores user latent vectors,
- Σ stores singular value importance,
- V^T stores movie latent vectors.

Chapter 5

The Sentiment Problem

Initially, the VADER sentiment analyzer was selected because:

- it is lightweight,
- rule-based,
- computationally efficient.

However, VADER performed poorly on nuanced, literary and emotionally reflective reviews.

To overcome this limitation, CineIQ integrated transformer-based contextual sentiment analysis using:

`distilbert-base-uncased-finetuned-sst-2-english`

The architecture evolved into a hybrid sentiment pipeline:

Simple reviews $\rightarrow$ VADER

Nuanced reviews $\rightarrow$ Transformer inference

This balanced semantic accuracy with computational efficiency.

Chapter 6

Adaptive Recommendation and Explainability

Cold-start users relied more heavily on semantic similarity, while experienced users received stronger collaborative weighting.

The final recommendation score was computed as:

$$\text{Final Score} = w_c C + w_s S + w_t T$$

where:

- C represents collaborative score,
- S represents semantic similarity,
- T represents sentiment score.

CineIQ additionally introduced explainability through recommendation reasons such as:

“Strong thematic similarity · predicted 4.5 · dark emotional tone”

Chapter 7

Deployment Engineering

Large precomputed matrices exceeded GitHub upload limits.

Several fallback strategies were explored:

- Git LFS,
- remote model hosting,
- dynamic recomputation.

GitHub Pages was unsuitable because CineIQ required:

- Python execution,
- Streamlit runtime,
- transformer inference,
- NumPy computation.

HuggingFace Spaces was selected because it provides:

- native Streamlit support,
- Python execution,
- transformer compatibility,
- free ML-oriented cloud hosting.

The deployment architecture became:

GitHub Repository → HuggingFace Spaces → Public CineIQ Application

Chapter 8

Conclusion

CineIQ evolved considerably during development.

The project began as a simple content recommender but gradually transformed into a hybrid cinematic intelligence system integrating:

- semantic vectorization,
- sparse matrix factorization,
- contextual NLP,
- adaptive weighting,
- explainability,
- deployment engineering.

Several technical failures ultimately improved the architecture:

- NumPy compatibility issues led to direct sparse linear algebra implementation.
- VADER limitations led to transformer integration.
- deployment constraints led to architectural simplification.

CineIQ therefore became not merely a recommender system, but an attempt to computationally model cinematic taste, emotional interpretation and user preference through mathematical and machine learning frameworks.